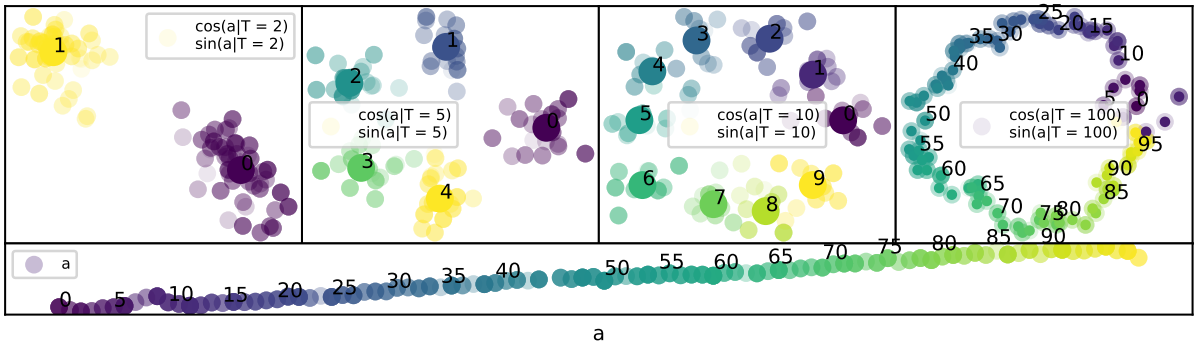


Figure 3. **Helix subspace visualized.** For GPT-J's layer 0 output, we project each of the numbers $a \in [0, 99]$ onto our fitted $T = [2, 5, 10, 100]$ helix subspace, and visualize it. In the top row, we plot $\sin(\frac{2\pi}{T_i}a)$ vs $\cos(\frac{2\pi}{T_i}a)$ for each $T_i \in T$ and plot all a congruent under $a \mod T$ in the same color and annotate their mean. The bottom row contains the linear component subplot.

projection of the residual stream into the helical subspace. When analyzing the columns of C, we find that the Fourier features increase in magnitude with period and are mostly orthogonal (Appendix C.1).

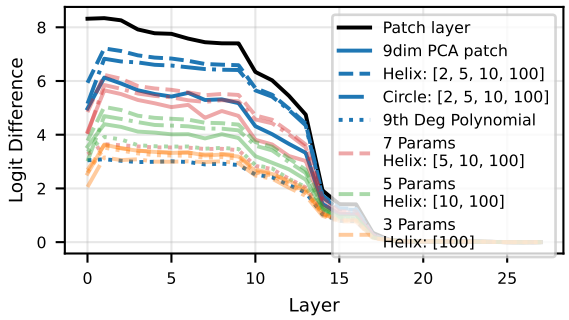


Figure 4. **Helix causal intervention results.** We use activation patching to causally determine if our fits preserve the information the model uses to compute $a + b$. We find that our helical and circular fits are strongly causally implicated, often outperforming the PCA baseline.

4.4. Evaluating the Quality of the Helical Fit

We want to causally demonstrate that the model actually uses the fitted helix. To do so, we employ activation patching. Activation patching isolates the contribution of specific model components towards answer tokens (Meng et al., 2022; Heimersheim & Nanda, 2024). Specifically, to evaluate the contribution of some residual stream h^l_a on the a token, we first store $h^l_{a,\text{clean}}$ when the model is run on a "clean" prompt $a + b$. We then run the model on the corrupted prompt $a' + b$ and store the model logits for the clean answer of $a + b$. Finally, we patch in the clean $h^l_{a,\text{clean}}$ on the corrupted prompt $a' + b$ and calculate $LD^l_a = \text{logit}_{\text{patched}}(a + b) - \text{logit}_{\text{corrupted}}(a + b)$, where LD^l_a is the logit difference for h^l_a. By averaging over 100 pairs of clean and corrupted prompts, we can evaluate h^l_a's ability to restore model behavior to the clean answer $a + b$. To reduce noise, all patching experiments only use prompts the model can successfully complete.

To leverage this technique, we follow Engels et al. (2024) and input our fit for $h^l_{a,\text{clean}}$ when patching. This allows us to causally determine if our fit preserves the information the model uses for the computation. We compare our k Fourier feature helical fit with four baselines: using the actual $h^l_{a,clean}$ (layer patch), the first $2k + 1$ PCA components of $h^l_{a,clean}$ (PCA), a circular fit with k Fourier components (circle), and a polynomial fit with basis terms $B(a) = [a, a^2, ...a^{2k+1}]$ (polynomial). For each value of k, we choose the combination of Fourier features that maximizes $\frac{1}{L}\sum_l LD^l_a$ as the best set of Fourier features.

In Fig. 4, we see that the helical fit is most performant against baselines, closely followed by the circular fit. This implies that Fourier features are predominantly used to compute addition. Surprisingly, the $k = 4$ full helical and circular fits dominate the strong PCA baseline and approach the effect of layer patching, which suggests that we have identified the correct "variables" of computation for addition. Additionally, we note a sharp jump between the fit for layer 0's input (the output of the embedding) and layer 1's input, aligning with evidence from Nikankin et al. (2024) that layer 0 is necessary for numerical processing.

In Appendix C.2, we provide evidence that Llama3.1-8B and Pythia-6.9B also use helical numerical representations. Additionally, we provide evidence that our fits are not overfitting by using a train-test split with no meaningful effect on our results. The helix functional form is not overly expressive, as a helix trained on a randomized order of a is not causally relevant. We also observe continuity when values of a that the helix was not trained on are projected into the

Table 1. **Performance of fits across tasks.** We calculate $\max_l LD_a^l$ for each fit across a variety of numerical tasks. While the helix fit is competitive, we find that it underperforms the PCA baseline on three tasks.

Task	Full	PCA	Helix	Circle	Poly
$a + b =$	**8.34**	6.13	**7.21**	6.83	3.09
$a - 23 =$	**7.45**	6.16	**7.05**	6.52	2.93
$a//5 =$	**5.48**	**5.24**	4.55	4.25	4.55
$a * 1.5 =$	**7.86**	**6.65**	5.16	4.85	4.84
$a \mod 2 =$	**0.98**	1.00	1.11	**1.12**	0.88
$x - a = 0, x =$	**6.70**	**4.77**	4.56	4.34	2.98

helical subspace. This satisfies the definition of a nonlinear feature manifold proposed by Olah & Jermyn (2024), and provides additional evidence for the argument of Engels et al. (2024) against the strongest form of the Linear Representation Hypothesis.

4.5. Is the Helix the Full Picture?

To identify if the helix sufficiently explains the structure of numbers in LLMs, we test on five additional tasks.

1. $a - 23$ for $a \in [23, 99]$
2. $a//5$ (integer division) for $a \in [0, 99]$
3. $a * 1.5$ for even $a \in [0, 98]$
4. $a \mod 2$ for $a \in [0, 99]$
5. If $x - a = 0$, what is $x =$ for $a \in [0, 99]$

For each task, we fit full helices with $T = [2, 5, 10, 100]$ and compare against baselines. In Table 1, we describe our results on these tasks by listing $\max_l LD_a^l$, which is the maximal causal power of each fit (full plot and additional task details in Appendix C.2). Notably, while the helix is causally relevant for all tasks, we see that it underperforms the PCA baseline on tasks 2, 3, and 5. This implies that there is potentially additional structure in numerical representations that helical fits do not capture. However, we are confident that the helix is used for addition. When ablating the helix dimensions from the residual stream (i.e. ablating $C^\dagger$ from h_a^l), performance is affected roughly as much as ablating h_a^l entirely (Fig. 21, Appendix C.1).

Thus, we conclude that LLMs use a helical representation of numbers to compute addition, although it is possible that additional structure is used for other tasks.

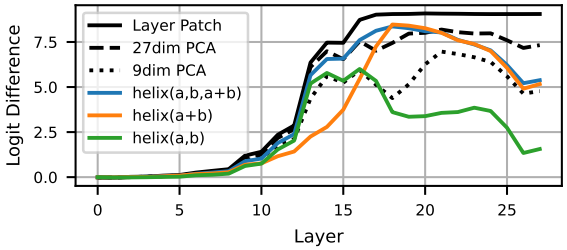


Figure 5. **Last token hidden states are well-modeled by** $\text{helix}(a + b)$**.** We use activation patching to show that $h_=^l$ for GPT-J is well modeled by helices, in particular $\text{helix}(a + b)$.

5. LLMs Use the Clock Algorithm to Compute Addition

5.1. Introducing the Clock Algorithm

Taking inspiration from Nanda et al. (2023a), we propose that LLMs manipulate helices to compute addition using the "Clock" algorithm.

To compute $a + b =$, GPT-J

1. Embeds a and b as helices on their own tokens.
2. A sparse set of attention heads, mostly in layers 9-14, move the a and b helices to the last token.
3. MLPs 14-18 manipulate these helices to create the helix $a + b$. A small number of attention heads help with this operation.
4. MLPs 19-27 and a few attention heads "read" from the $a + b$ helix and output to model logits.

Since we have already shown that models represent a and b as helices (Appendix C.2), we provide evidence for the last three steps in this section. In Fig. 5 we observe that last token hidden states are well modeled by $h_=^l = \text{helix}(a, b, a + b)$, where $\text{helix}(x, y)$ is shorthand to denote $\text{helix}(x) + \text{helix}(y)$. Remarkably, despite only using 9 parameters, at some layers $\text{helix}(a + b)$ fits last token hidden states better than a 27 dimensional PCA. The $a + b$ helix having such causal power implies it is at the heart of the computation.

In Appendix D, we show that other LLMs also use $\text{helix}(a + b)$. Since the crux of the Clock algorithm is computing the answer helix for $a + b$, we take this as compelling evidence that all three models use the Clock algorithm. However, we would like to understand how specific LLM components implement the algorithm. To do so, we focus on GPT-J.

In Fig. 6, we use activation patching to determine which last token MLP and attention layers are most influential for

the final result. We also present path patching results, which isolates how much components directly contribute to logits. For example, MLP18's total effect (TE, activation patching) includes both its indirect effect (IE), or how MLP18's output is used by downstream components like MLP19, and its direct effect (DE, path patching), or how much MLP18 directly boosts the answer logit.[1] In Fig. 6, we see that MLPs dominate direct effect.

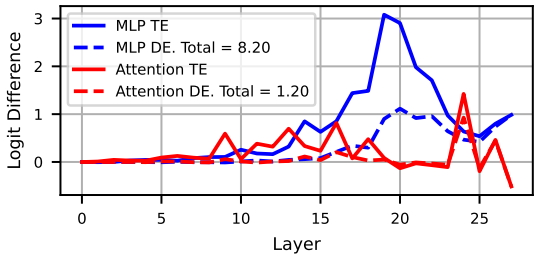


Figure 6. **MLPs drive computation of $a + b$.** By using activation and path patching, we find that MLPs are most implicated in constructing the final answer, along with early attention layers.

We now investigate specific attention heads, MLPs, and individual neurons.

5.2. Investigating Attention Heads

In GPT-J, every attention layer is the sum of 16 attention heads whose outputs are concatenated. We activation and path patch each attention head on the last token and rank them by total effect (TE). To determine the minimal set of attention heads required, we activation patch k attention heads at once, and find the minimum k such that their combined total effect approximates patching in all attention heads. In Appendix D.1, we see that patching $k = 17$ heads achieves 80% of the effect of patching in all 448 attention heads, and we choose to round up to $k = 20$ heads (83.9% of effect).

Since attention heads are not as influential as MLPs in Fig. 6, we hypothesize that they primarily serve two roles: 1) moving the a, b helices to the last token to be processed by downstream components (a, b heads) and 2) outputting the $a + b$ helix directly to logits ($a + b$ heads). Some mixed heads output all three $a, b,$ and $a + b$ helices. We aim to categorize as few attention heads as mixed as possible.

To categorize attention heads, we turn to two metrics. $c_{a,b}$ is the confidence that a certain head is an a, b head, which we quantify with $c_{a,b} = (1 - \frac{\text{DE}}{\text{TE}})\frac{\text{helix}(a,b)}{\text{helix}(a,b,a+b)}$. The first term represents the fractional indirect effect of the attention head, and the second term represents the head's total effect recoverable by just using the a, b helices instead of $\text{helix}(a, b, a + b)$. Similarly, we calculate c_{a+b} as the confidence the head is an $a + b$ head, using $c_{a+b} = \frac{\text{DE}}{\text{TE}}\frac{\text{helix}(a+b)}{\text{helix}(a,b,a+b)}$.

We sort the $k = 20$ heads by $c = \max(c_{a,b}, c_{a+b})$. If a head is an $a+b$ head, we model its output using the $a+b$ helix and allow it only to output to logits (no impact on downstream components). If a head is an a, b head, we restrict it to outputting $\text{helix}(a, b)$. For $m = [1, 20]$, we allow m heads with the lowest c to be mixed heads, and categorize the rest as a, b or $a + b$ heads. We find that categorizing $m = 4$ heads as mixed is sufficient to achieve almost 80% of the effect of using the actual outputs of all $k = 20$ heads. Thus, most important attention heads obey our categorization. We list some properties of each head type below.

- a, b heads (11/20): In layers 9-14 (but two heads in $l = 16, 18$), attend to the a, b tokens, and output a, b helices which are used mostly by downstream MLPs.
- $a + b$ heads (5/20): In layers 24-26 (but one head in layer 19), attend to the last token, take their input from preceding MLPs, and output the $a + b$ helix to logits.
- Mixed heads (4/20): In layers 15-18, attend to the $a, b,$ and $a + b$ tokens, receive input from a, b attention heads and previous MLPs, and output the $a, b,$ and $a + b$ helices to downstream MLPs.

For evidence of these properties refer to Appendix D.1. Notably, only mixed heads are potentially involved in creating the $a + b$ helix, which is the crux of the computation, justifying our conclusion from Fig. 6 that MLPs drive addition.

5.3. Looking at MLPs

GPT-J seems to predominantly rely on last token MLPs to compute $a + b$. To identify which MLPs are most important, we first sort MLPs by total effect, and patch in $k = [1, L = 28]$ MLPs to find the smallest k such that we achieve 95% of the effect of patching in all L MLPs. We use a sharper 95% threshold because MLPs dominate computation and because there are so few of them. Thus, we use $k = 11$ MLPs in our circuit, specifically MLPs 14-27, with the exception of MLPs 15, 24, and 25 (see Appendix D.2 for details).

We hypothesize that MLPs serve two functions: 1) reading from the a, b helices to create the $a + b$ helix and 2) reading from the $a+b$ helix to output the answer in model logits. We make this distinction using two metrics: $\text{helix}(a + b)$/TE, or the total effect of the MLP recoverable from modeling its output with $\text{helix}(a + b)$, and DE/TE ratio. In Fig. 7, we see that the outputs of MLPs 14-18 are progressively better modeled using $\text{helix}(a + b)$. Most of their effect is indirect and thus their output is predominantly used by downstream components. At layer 19, $\text{helix}(a + b)$ becomes a worse

[1] For more on path patching, we refer readers to Goldowsky-Dill et al. (2023); Wang et al. (2023)